

Práctica 4: Planificación Automática

Inteligencia Artificial

Grado en Ingeniería Informática – Ingeniería del Software
Departamento de Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla

Curso 2025-26

Contenido

- 1 Repaso: Planificación Automática
- 2 Repaso: Búsqueda y Heurísticas
- 3 Ejercicio 1: Mundo de los Bloques
- 4 Ejercicio 2: Dominio Depot
- 5 Ejercicio 3: Sokoban

¿Qué es la Planificación Automática?

Definición

La **planificación automática** es el campo de la IA que estudia cómo anticipar el resultado de las posibles acciones para seleccionarlas y organizarlas de manera que permitan alcanzar un objetivo.

Tipos de planificadores:

Específicos del dominio Optimizados para un problema concreto; difíciles de generalizar.

Independientes del dominio No usan conocimiento específico; aplicables en principio a cualquier dominio.

Configurables Independientes del dominio, pero aceptan información específica como entrada.

En esta práctica usaremos un planificador **independiente del dominio**: Fast Downward.

Planificación Clásica: Las 8 Restricciones

En la **planificación clásica** se asume que el dominio satisface:

- 1 Sistema **finito**: número finito de estados y acciones.
- 2 **Completamente observable**: conocimiento completo del estado actual.
- 3 **Determinista**: mismo resultado al aplicar la misma acción al mismo estado.
- 4 **Estático**: el estado solo cambia al aplicar una acción.
- 5 **Consecución de estados objetivo**: el objetivo es llegar a ciertos estados.
- 6 **Planes secuenciales**: la solución es una secuencia de acciones.
- 7 **Tiempo implícito**: las acciones se consideran instantáneas.
- 8 **Planificación fuera de línea**: el entorno no cambia mientras se planifica.

Un problema de planificación $P = (H, A, I, G)$ se especifica mediante:

- H Conjunto finito de **hechos** (predicados aplicados a constantes).
- A Conjunto finito de **acciones**, cada una con:
 - **Precondiciones**: hechos que deben cumplirse para poder aplicarla.
 - **Lista de borrado**: hechos que dejan de cumplirse al aplicarla.
 - **Lista de adición**: hechos que pasan a cumplirse al aplicarla.
- I **Estado inicial**: hechos que se cumplen inicialmente.
- G **Objetivo**: hechos que se quiere que se cumplan.

Hipótesis del mundo cerrado

Todo hecho no incluido en un estado se asume **falso**.

Esquemas de Acciones: el Mundo de los Bloques

Predicados: $\text{sobre_la_mesa}(b)$, $\text{sobre}(b_1, b_2)$, $\text{agarrado}(b)$, $\text{brazo_libre}()$, $\text{despejado}(b)$

agarrar(b)

- *Pre:* $\text{sobre_la_mesa}(b)$, $\text{despejado}(b)$, $\text{brazo_libre}()$
- *Del:* $\text{sobre_la_mesa}(b)$, $\text{despejado}(b)$, $\text{brazo_libre}()$
- *Add:* $\text{agarrado}(b)$

desapilar(b_1, b_2)

- *Pre:* $\text{sobre}(b_1, b_2)$, $\text{despejado}(b_1)$, $\text{brazo_libre}()$
- *Del:* $\text{sobre}(b_1, b_2)$, $\text{despejado}(b_1)$, $\text{brazo_libre}()$
- *Add:* $\text{agarrado}(b_1)$, $\text{despejado}(b_2)$

bajar(b)

- *Pre:* $\text{agarrado}(b)$
- *Del:* $\text{agarrado}(b)$
- *Add:* $\text{sobre_la_mesa}(b)$, $\text{despejado}(b)$, $\text{brazo_libre}()$

apilar(b_1, b_2)

- *Pre:* $\text{agarrado}(b_1)$, $\text{despejado}(b_2)$
- *Del:* $\text{agarrado}(b_1)$, $\text{despejado}(b_2)$
- *Add:* $\text{sobre}(b_1, b_2)$, $\text{despejado}(b_1)$, $\text{brazo_libre}()$

PDDL

Planning Domain Definition Language: implementación práctica del formalismo STRIPS. Es el formato de entrada estándar de todos los planificadores modernos.

Unified Planning (biblioteca Python) permite:

- Leer dominios e instancias en formato **PDDL**.
- Construir dominios e instancias mediante **código Python**.
- Llamar a distintos **planificadores** (Fast Downward, etc.) con una interfaz unificada.
- Exportar un dominio construido en Python de vuelta a **PDDL**.

Instalación completa: <https://codeshare.io/iais-pa>

```
pip install unified-planning[fast-downward]
```

Resolución mediante Búsqueda en el Espacio de Estados

La solución de un problema de planificación se puede encontrar buscando un **camino en el grafo de estados**:

- **Vértices**: estados posibles del problema.
- **Arcos**: acciones que transforman un estado en otro.
- **Objetivo**: camino desde s_I hasta algún estado $s \in S_G$.

Problema: el número de estados crece exponencialmente.

Bloques	Estados
3	22
5	866
7	65.990
9	8.145.730

⇒ Es imprescindible usar **heurísticas** para guiar la búsqueda.

El Algoritmo A^*

A^* guía la búsqueda usando una **función heurística** $h(s)$ que estima el coste mínimo desde s hasta el objetivo.

Para cada nodo abierto s se calcula:

$$f(s) = g(s) + h(s)$$

donde $g(s)$ es el coste del camino encontrado hasta s y $h(s)$ es la estimación heurística.

Admisibilidad $\Rightarrow$ optimalidad

Si h es **admisible** ($h(s) \leq$ coste real para todo s), entonces A^* es **completo** y **óptimo**.

En Fast Downward: `astar(hmax())` ejecuta A^* con la heurística $h^{\text{máx}}$.

La Heurística $h^{\text{máx}}$

Se basa en la **relajación del borrado**: se eliminan las listas de borrado de todas las acciones, de forma que aplicar una acción solo puede *añadir* hechos, nunca eliminarlos.

$h^{\text{máx}}$ asume que para lograr un *conjunto* de objetivos basta con lograr el objetivo **más costoso**:

$$h^{\text{máx}}(s, G) = \begin{cases} 0 & \text{si } G = \{g\} \text{ y } g \in s, \\ \min_{\substack{a \in A \\ g \in \text{add}(a)}} (h^{\text{máx}}(s, \text{pre}(a)) + c_a) & \text{si } G = \{g\} \text{ y } g \notin s, \\ \max_{g \in G} h^{\text{máx}}(s, \{g\}) & \text{si } |G| > 1. \end{cases}$$

Propiedad

$h^{\text{máx}}$ es admisible $\Rightarrow A^* + h^{\text{máx}}$ garantiza planes **óptimos**.

Ejercicio 1: Descripción

Contexto teórico

El **mundo de los bloques** es el dominio de referencia de la planificación clásica. Su dominio PDDL ya está disponible en `dominio_mundo_bloques.pddl`.

Objetivo del ejercicio:

- 1 **Leer** el dominio desde el fichero PDDL.
- 2 **Completar** la función `crea_instancia_mundo_bloques(N)` que construye una instancia con N bloques $B_0, \dots, B_{N-1}$ apilados en ese orden, con el objetivo de invertir la pila (B_{N-1} abajo, B_0 arriba).
- 3 **Resolver** la instancia con el mayor N posible usando Fast Downward.

Conexión con la teoría: el estado inicial y el objetivo son conjuntos de hechos STRIPS $\Rightarrow$ **estado inicial** e I , **objetivo** G de la tupla $P = (H, A, I, G)$.

Ejercicio 1.1 — Leer el dominio PDDL

```
from unified_planning.io import PDDLReader

lector_PDDL = PDDLReader()
dominio_mundo_bloques = lector_PDDL.parse_problem(
    "dominio_mundo_bloques.pddl", None)

print(dominio_mundo_bloques)
```

Nota: pasar `None` como segundo argumento indica que solo se está leyendo el *dominio*, sin una instancia concreta. La salida muestra los predicados y los esquemas de acciones en lenguaje Unified Planning.

Ejercicio 1.2 — Estado inicial y objetivo

Para la instancia con N bloques $B_0, \dots, B_{N-1}$:

Estado inicial (pila B_0 abajo, B_{N-1} arriba):

- $\text{sobre_la_mesa}(B_0)$ — B_0 está en la mesa.
- $\text{sobre}(B_i, B_{i-1})$ para $i = 1, \dots, N-1$ — cada bloque sobre el anterior.
- $\text{brazo_libre}()$ — el brazo no tiene ningún bloque.
- $\text{despejado}(B_{N-1})$ — solo el bloque superior está libre.

Objetivo (pila invertida, B_{N-1} abajo):

- $\text{sobre_la_mesa}(B_{N-1})$
- $\text{sobre}(B_{i-1}, B_i)$ para $i = N-1, \dots, 1$ — pila en orden inverso.

Ejercicio 1.2 — Completa el código (1/2)

```
from unified_planning.shortcuts import *

def crea_instancia_mundo_bloques(N):
    instancia = dominio_mundo_bloques.clone()
    # Anadir los objetos de la instancia
    tipo_objeto = instancia.user_type('object')
    bloques = ___ # lista de N objetos B0..B(N-1)
    instancia.___
    # Obtener los fluentes del dominio
    sobre_la_mesa = dominio_mundo_bloques.fluent('sobre_la_mesa')
    sobre = dominio_mundo_bloques.fluent('sobre')
    brazo_libre = dominio_mundo_bloques.fluent('brazo_libre')
    despejado = dominio_mundo_bloques.fluent('despejado')
    # ...
```

Objetivo: Crear una instancia con N bloques y obtener los fluentes del dominio.

- Clonar el dominio para crear una nueva instancia
- Crear lista de N objetos de tipo object (bloques $B_0..B_{N-1}$)
- Obtener referencias a los 4 fluentes del dominio

Ejercicio 1.2 — Implementación (1/2)

```
from unified_planning.shortcuts import *

def crea_instancia_mundo_bloques(N):
    instancia = dominio_mundo_bloques.clone()
    # Anadir los objetos de la instancia
    tipo_objeto = instancia.user_type('object')
    bloques = [Object(f'B{i}', tipo_objeto) for i in range(N)]
    instancia.add_objects(bloques)
    # Obtener los fluentes del dominio
    sobre_la_mesa = dominio_mundo_bloques.fluent('sobre_la_mesa')
    sobre = dominio_mundo_bloques.fluent('sobre')
    brazo_libre = dominio_mundo_bloques.fluent('brazo_libre')
    despejado = dominio_mundo_bloques.fluent('despejado')
    # ...
```

Solución: Crear una lista de N objetos usando comprensión de listas.

- `Object(f'B{i}', tipo_objeto)` crea un objeto con nombre B0, B1, etc.
- La comprensión de lista genera todos los bloques en una sola línea

Ejercicio 1.2 — Completa el código (2/2)

```
# Estado inicial
instancia.set_initial_value(___, True) # B0 en la mesa
for i in range(1, N):
    instancia.set_initial_value(___, True) # Bi sobre B(i-1)
instancia.set_initial_value(___, True) # brazo libre
instancia.set_initial_value(___, True) # B(N-1) despejado
# Objetivo
instancia.add_goal(___) # B(N-1) en la mesa
for i in range(N - 1, 0, -1):
    instancia.add_goal(___) # B(i-1) sobre Bi
return instancia
```

Objetivo: Definir estado inicial (torre) y objetivo (torre invertida).

- **Estado inicial:** B0 en mesa, resto apilados (B_i sobre $B_i - 1$), brazo libre, $B_N - 1$ despejado
- **Objetivo:** $B_N - 1$ en mesa, resto apilados al revés ($B_i - 1$ sobre B_i)

Ejercicio 1.2 — Implementación (2/2)

```
# Estado inicial
instancia.set_initial_value(sobre_la_mesa(bloques[0]), True)
for i in range(1, N):
    instancia.set_initial_value(sobre(bloques[i], bloques[i-1]),
                                True)
instancia.set_initial_value(brazo_libre(), True)
instancia.set_initial_value(despejado(bloques[N-1]), True)
# Objetivo
instancia.add_goal(sobre_la_mesa(bloques[N-1]))
for i in range(N - 1, 0, -1):
    instancia.add_goal(sobre(bloques[i-1], bloques[i]))
return instancia
```

Solución: Aplicar los predicados correctos con los bloques adecuados.

- **Estado inicial:** pila con bloques[0] abajo y bloques[N-1] arriba
- **Objetivo:** pila invertida con bloques[N-1] abajo y bloques[0] arriba

Ejercicio 1.3 — Completa el código

```
planificador = OneshotPlanner(name=___)

problema_mundo_bloques = crea_instancia_mundo_bloques(___ )
resultado = planificador.___(___)
print(resultado)

if ___ is not None:
    print(f"Plan encontrado con "
          f"{len(___)} pasos.")
```

Ejercicio 1.3 — Implementación

```
planificador = OneshotPlanner(name='fast-downward')

problema_mundo_bloques = crea_instancia_mundo_bloques(100)
resultado = planificador.solve(problema_mundo_bloques)
print(resultado)

if resultado.plan is not None:
    print(f"Plan encontrado con "
          f"{len(resultado.plan.actions)} pasos.")
```

Ejercicio 2: Descripción

Contexto

Dominio de la **Competición Internacional de Planificación** (ICAPS 2002).
Combina el mundo de los bloques con distribución logística:

- **Lugares**: almacenes (*Depot*) y distribuidores (*Distributor*).
- **Camiones** (capacidad infinita): transportan cajas entre lugares.
- **Palés**: superficies fijas en cada lugar sobre las que se apilan cajas.
- **Polipastos**: uno por lugar, para apilar/desapilar cajas.

Objetivo: completar los efectos de las 5 acciones del dominio.

Ejercicio 2: Jerarquía de tipos y predicados

Jerarquía de tipos:

Tipo	Supertipo
Place	—
Depot, Distributor	Place
Locatable	—
Truck, Hoist, Surface	Locatable
Pallet, Crate	Surface

Predicados:

- $AT(x, y)$: el ubicable x está en el lugar y .
- $ON(x, y)$: la caja x está sobre la superficie y .
- $IN(x, y)$: la caja x está en el camión y .
- $LIFTING(x, y)$: el polipasto x está levantando la caja y .
- $AVAILABLE(x)$: el polipasto x está disponible.
- $CLEAR(x)$: la superficie x está despejada.

Ejercicio 2: Acción DRIVE

DRIVE(x, y, z): el camión x va del lugar y al lugar z (todos los lugares están conectados entre sí).

Precondiciones:

- $AT(x, y)$

Efectos:

- $AT(x, y)$
- + $AT(x, z)$

Conexión teórica: los efectos negativos corresponden a la *lista de borrado* de STRIPS; los positivos, a la *lista de adición*. Solo cambia lo que es necesario cambiar — principio de **representación factorizada**.

Ejercicio 2 — DRIVE: Completa el código

```
drive = InstantaneousAction('DRIVE', x=Truck, y=Place, z=Place)
x = drive.x; y = drive.y; z = drive.z

for hecho in [__]:          # Precondiciones
    drive.add_precondition(hecho)
for hecho in [__]:          # Lista de borrado
    drive.add_effect(hecho, False)
for hecho in [__]:          # Lista de adición
    drive.add_effect(hecho, True)
```

Significado: Mover camión x del lugar y al lugar z .

- **Precond.**: el camión debe estar en el origen
- **Efectos** $-$: el camión ya no está en el origen
- **Efectos** $+$: el camión ahora está en el destino

Ejercicio 2 — DRIVE: Implementación

```
drive = InstantaneousAction('DRIVE', x=Truck, y=Place, z=Place)
x = drive.x; y = drive.y; z = drive.z

for hecho in [at(x, y)]:
    drive.add_precondition(hecho)
for hecho in [at(x, y)]:
    drive.add_effect(hecho, False)
for hecho in [at(x, z)]:
    drive.add_effect(hecho, True)
```

Significado: Mover camión x del lugar y al lugar z .

- **Precond.:** el camión debe estar en el origen
- **Efectos $-$:** el camión ya no está en el origen
- **Efectos $+$:** el camión ahora está en el destino

Ejercicio 2: Acciones LIFT y DROP

LIFT(x, y, z, p): el polipasto x levanta la caja y que estaba sobre la superficie z en el lugar p .

Precondiciones: $AT(x, p)$, $AVAILABLE(x)$,
 $AT(y, p)$, $ON(y, z)$, $CLEAR(y)$

Efectos:

– $AT(y, p)$, $CLEAR(y)$, $AVAILABLE(x)$,
 $ON(y, z)$
+ $LIFTING(x, y)$, $CLEAR(z)$

DROP(x, y, z, p): el polipasto x deja la caja y sobre la superficie z en el lugar p .

Precondiciones: $AT(x, p)$, $AT(z, p)$,
 $CLEAR(z)$, $LIFTING(x, y)$

Efectos:

– $LIFTING(x, y)$, $CLEAR(z)$
+ $AVAILABLE(x)$, $AT(y, p)$,
 $CLEAR(y)$, $ON(y, z)$

Ejercicio 2 — LIFT: Completa el código

```
lift = InstantaneousAction('LIFT', x=Hoist, y=Crate,
                           z=Surface, p=Place)
x = lift.x;  y = lift.y;  z = lift.z;  p = lift.p

for hecho in [__]:      # Precondiciones
    lift.add_precondition(hecho)
for hecho in [__]:      # Lista de borrado
    lift.add_effect(hecho, False)
for hecho in [__]:      # Lista de adición
    lift.add_effect(hecho, True)
```

Significado: Polipasto x levanta caja y de superficie z en lugar p .

- **Precond.**: polipasto en p , disponible; caja en p , sobre z , despejada
- **Efectos** $-$: caja ya no en p , no despejada, no sobre z ; polipasto no disponible
- **Efectos** $+$: polipasto levantando caja; superficie z despejada

Ejercicio 2 — LIFT: Implementación

```
lift = InstantaneousAction('LIFT', x=Hoist, y=Crate,
                           z=Surface, p=Place)
x = lift.x;  y = lift.y;  z = lift.z;  p = lift.p

for hecho in [at(x,p), available(x), at(y,p), on(y,z), clear(y)]:
    lift.add_precondition(hecho)
for hecho in [at(y,p), clear(y), available(x), on(y,z)]:
    lift.add_effect(hecho, False)
for hecho in [lifting(x,y), clear(z)]:
    lift.add_effect(hecho, True)
```

Significado: Polipasto x levanta caja y de superficie z en lugar p .

- **Precond.:** polipasto en p , disponible; caja en p , sobre z , despejada
- **Efectos $-$:** caja ya no en p , no despejada, no sobre z ; polipasto no disponible
- **Efectos $+$:** polipasto levantando caja; superficie z despejada

Ejercicio 2: Acciones LOAD y UNLOAD

LOAD(x, y, z, p): el polipasto x carga la caja y en el camión z en el lugar p .

Precondiciones:

$AT(x, p)$, $AT(z, p)$, $LIFTING(x, y)$

Efectos:

– $LIFTING(x, y)$
+ $IN(y, z)$, $AVAILABLE(x)$

UNLOAD(x, y, z, p): el polipasto x descarga la caja y del camión z en el lugar p .

Precondiciones:

$AT(x, p)$, $AT(z, p)$,
 $AVAILABLE(x)$, $IN(y, z)$

Efectos:

– $IN(y, z)$, $AVAILABLE(x)$
+ $LIFTING(x, y)$

Ejercicio 2 — LOAD: Completa el código

```
load = InstantaneousAction('LOAD', x=Hoist, y=Crate,
                           z=Truck, p=Place)
x = load.x; y = load.y; z = load.z; p = load.p
for hecho in [__]:      # Precondiciones
    load.add_precondition(hecho)
for hecho in [__]:      # Lista de borrado
    load.add_effect(hecho, False)
for hecho in [__]:      # Lista de adición
    load.add_effect(hecho, True)
```

Significado: Polipasto x carga caja y en camión z en lugar p .

- **Precond.:** polipasto en p , camión en p , polipasto levantando la caja
- **Efectos** —: polipasto ya no levanta la caja
- **Efectos** +: caja dentro del camión; polipasto disponible

Ejercicio 2 — LOAD: Implementación

```
load = InstantaneousAction('LOAD', x=Hoist, y=Crate,
                           z=Truck, p=Place)
x = load.x; y = load.y; z = load.z; p = load.p
for hecho in [at(x,p), at(z,p), lifting(x,y)]:
    load.add_precondition(hecho)
for hecho in [lifting(x,y)]:
    load.add_effect(hecho, False)
for hecho in [in_(y,z), available(x)]:
    load.add_effect(hecho, True)
```

Significado: Polipasto x carga caja y en camión z en lugar p .

- **Precond.:** polipasto en p , camión en p , polipasto levantando la caja
- **Efectos** —: polipasto ya no levanta la caja
- **Efectos** +: caja dentro del camión; polipasto disponible

Ejercicio 2 — UNLOAD: Completa el código

```
unload = InstantaneousAction('UNLOAD', x=Hoist, y=Crate,
                              z=Truck, p=Place)
x = unload.x;  y = unload.y;  z = unload.z;  p = unload.p
for hecho in [__]:          # Precondiciones
    unload.add_precondition(hecho)
for hecho in [__]:          # Lista de borrado
    unload.add_effect(hecho, False)
for hecho in [__]:          # Lista de adición
    unload.add_effect(hecho, True)
```

Significado: Polipasto x descarga caja y del camión z en lugar p .

- **Precond.**: polipasto en p , camión en p , polipasto disponible, caja en el camión
- **Efectos** $-$: caja ya no en el camión; polipasto deja de estar disponible
- **Efectos** $+$: polipasto levantando la caja

Ejercicio 2 — UNLOAD: Implementación

```
unload = InstantaneousAction('UNLOAD', x=Hoist, y=Crate,
                              z=Truck, p=Place)
x = unload.x; y = unload.y; z = unload.z; p = unload.p
for hecho in [at(x,p), at(z,p), available(x), in_(y,z)]:
    unload.add_precondition(hecho)
for hecho in [in_(y,z), available(x)]:
    unload.add_effect(hecho, False)
for hecho in [lifting(x,y)]:
    unload.add_effect(hecho, True)
```

Significado: Polipasto x descarga caja y del camión z en lugar p .

- **Precond.:** polipasto en p , camión en p , polipasto disponible, caja en el camión
- **Efectos** $-$: caja ya no en el camión; polipasto deja de estar disponible
- **Efectos** $+$: polipasto levantando la caja

Ejercicio 2 — Resolver instancias Depot

```
from unified_planning.io import PDDLWriter, PDDLReader

# Exportar el dominio construido a PDDL
escritor_PDDL = PDDLWriter(dominio_depot)
escritor_PDDL.write_domain('dominio_depot.pddl')

# Leer una instancia del problema
lector_PDDL = PDDLReader()
problema_depot = lector_PDDL.parse_problem(
    'dominio_depot.pddl', 'Depot/pfile1')

# Resolver con Fast Downward
planificador = OneshotPlanner(name='fast-downward')
resultado = planificador.solve(problema_depot)
print(resultado)
```

Experimento: intentar resolver las instancias pfile1, pfile2, ... de la carpeta Depot/. ¿Hasta cuál puede llegar el planificador?

Ejercicio 3: Descripción

Sokoban es un videojuego de puzle: el objetivo es empujar **piedras** (cajas) hasta sus **posiciones objetivo** en un almacén.

Reglas:

- Solo se pueden **empujar** objetos, no tirar de ellos.
- Solo se puede empujar **un objeto a la vez**.
- Hay que evitar **puntos muertos**: situaciones donde el puzle se vuelve irresoluble.

Objetivo del ejercicio:

- 1 Construir el **dominio** de planificación para Sokoban.
- 2 Usar $A^* + h^{\max}$ para resolver los puzles de la carpeta Sokoban/ con el **menor número de movimientos de empuje**.

Ejercicio 3: Tipos y Predicados

Tipos de objetos:

- `thing`: cosa en general. Subtipos: `player`, `stone`.
- `location`: casilla del tablero.
- `direction`: dirección de movimiento (norte, sur, este, oeste).

Predicados:

- `CLEAR(l)`: la localización *l* no contiene ninguna cosa.
- `AT(t, l)`: la cosa *t* está en la localización *l*.
- `AT-GOAL(s)`: la piedra *s* está en su posición objetivo.
- `IS-GOAL(l)`: *l* es una localización objetivo.
- `IS-NONGOAL(l)`: *l* no es una localización objetivo.
- `MOVE-DIR(f, t, d)`: desde *f* se puede ir a *t* moviéndose en dirección *d*.

Ejercicio 3: Acciones

MOVE(p, f, t, d): el jugador p se mueve de f a t (libre) en dirección d .

Precondiciones:

$AT(p, f)$, $CLEAR(t)$, $MOVE-DIR(f, t, d)$

Efectos:

– $AT(p, f)$, $CLEAR(t)$

+ $AT(p, t)$, $CLEAR(f)$

PUSH-TO-NONGOAL / PUSH-TO-GOAL(p, pos, s, f, t, d): el jugador p en pos empuja la piedra s de f a t (libre, no objetivo / objetivo) en dirección d .

La diferencia entre ambas acciones está en si la casilla destino **es o no un objetivo** y en el efecto sobre el predicado $AT-GOAL$.

Ejercicio 3 — MOVE: Completa el código

```
move = InstantaneousAction('MOVE',  
    p=player, f=location, t=location, d=direction)  
p = move.p; f = move.f; t = move.t; d = move.d  
  
for hecho in [__]:      # Precondiciones  
    move.add_precondition(hecho)  
for hecho in [__]:      # Lista de borrado  
    move.add_effect(hecho, False)  
for hecho in [__]:      # Lista de adicion  
    move.add_effect(hecho, True)
```

Significado: Jugador p se mueve de f a t en dirección d .

- **Precond.:** jugador en f , casilla t libre, conexión en dirección d
- **Efectos $-$:** jugador ya no en f , casilla t deja de estar libre
- **Efectos $+$:** jugador ahora en t , casilla f queda libre

Ejercicio 3 — MOVE: Implementación

```
move = InstantaneousAction('MOVE',
    p=player, f=location, t=location, d=direction)
p = move.p;  f = move.f;  t = move.t;  d = move.d

for hecho in [at(p,f), clear(t), move_dir(f,t,d)]:
    move.add_precondition(hecho)
for hecho in [at(p,f), clear(t)]:
    move.add_effect(hecho, False)
for hecho in [at(p,t), clear(f)]:
    move.add_effect(hecho, True)
```

Ejercicio 3 — PUSH-TO-NONGOAL: Completa el código (1/2)

```
push_to_nongoal = InstantaneousAction(  
    'PUSH-TO-NONGOAL',  
    p=player, pos=location, s=stone,  
    f=location, t=location, d=direction)  
p = push_to_nongoal.p; pos = push_to_nongoal.pos  
s = push_to_nongoal.s; f = push_to_nongoal.f  
t = push_to_nongoal.t; d = push_to_nongoal.d
```

Ejercicio 3 — PUSH-TO-NONGOAL: Completa el código (2/2)

```
for hecho in [__]:      # Precondiciones
    push_to_nongoal.add_precondition(hecho)
# Borrado: el jugador sale de pos, la piedra sale de f,
#         t deja de estar libre, at-goal(s) se borra si estaba
for hecho in [__]:
    push_to_nongoal.add_effect(hecho, False)
# Adicion: el jugador llega a f, la piedra llega a t,
#         pos queda libre
for hecho in [__]:
    push_to_nongoal.add_effect(hecho, True)
```

Significado: Jugador en *pos* empuja piedra *s* de *f* a *t* (no-objetivo).

- **Precond.:** jugador en *pos*, piedra en *f*, *t* libre, conexiones *d*, *t* no-objetivo
- **Efectos** —: jugador sale de *pos*, piedra sale de *f*, *t* ocupada, *at_goal(s)* borrado
- **Efectos** +: jugador en *f*, piedra en *t*, *pos* libre

Ejercicio 3 — PUSH-TO-NONGOAL: Implementación

```
for hecho in [at(p,pos), at(s,f), clear(t),
              move_dir(pos,f,d), move_dir(f,t,d),
              is_nongoal(t)]:
    push_to_nongoal.add_precondition(hecho)
for hecho in [at(p,pos), at(s,f), clear(t), at_goal(s)]:
    push_to_nongoal.add_effect(hecho, False)
for hecho in [at(p,f), at(s,t), clear(pos)]:
    push_to_nongoal.add_effect(hecho, True)
```

Nota: `at_goal(s)` se elimina porque la piedra sale de la casilla objetivo que pudiera estar ocupando (si la tuviera). La acción `PUSH-TO-GOAL` hace lo contrario: *añade* `at_goal(s)`.

Ejercicio 3 — PUSH-TO-GOAL: Completa el código (1/2)

```
push_to_goal = InstantaneousAction(  
    'PUSH-TO-GOAL',  
    p=player, pos=location, s=stone,  
    f=location, t=location, d=direction)  
p = push_to_goal.p;  pos = push_to_goal.pos  
s = push_to_goal.s;  f = push_to_goal.f  
t = push_to_goal.t;  d = push_to_goal.d
```

Ejercicio 3 — PUSH-TO-GOAL: Completa el código (2/2)

```
for hecho in [__]:      # Precondiciones (igual pero is_goal(t))
    push_to_goal.add_precondition(hecho)
# Borrado: el jugador sale de pos, la piedra sale de f,
#         t deja de estar libre
for hecho in [__]:
    push_to_goal.add_effect(hecho, False)
# Adicion: el jugador llega a f, la piedra llega a t,
#         pos queda libre, at_goal(s) se activa
for hecho in [__]:
    push_to_goal.add_effect(hecho, True)
```

Significado: Jugador en *pos* empuja piedra *s* de *f* a objetivo *t*.

- **Precond.:** como PUSH-TO-NONGOAL pero *t* es objetivo (*is_goal*)
- **Efectos –:** jugador sale de *pos*, piedra sale de *f*, *t* ocupada
- **Efectos +:** jugador en *f*, piedra en *t*, *pos* libre, *at_goal(s)* activado

Ejercicio 3 — PUSH-TO-GOAL: Implementación

```
for hecho in [at(p,pos), at(s,f), clear(t),
              move_dir(pos,f,d), move_dir(f,t,d),
              is_goal(t)]:
    push_to_goal.add_precondition(hecho)
for hecho in [at(p,pos), at(s,f), clear(t)]:
    push_to_goal.add_effect(hecho, False)
for hecho in [at(p,f), at(s,t), clear(pos), at_goal(s)]:
    push_to_goal.add_effect(hecho, True)
```

Comparativa con PUSH-TO-NONGOAL:

- La precondition `is_nongoal(t)` se reemplaza por `is_goal(t)`.
- En el borrado ya no aparece `at_goal(s)` (la piedra puede que no estuviera en un objetivo antes).
- En la adición se incluye `at_goal(s)`.

Ejercicio 3 — $A^* + h^{\text{máx}}$: Completa el código (1/2)

La métrica minimiza los movimientos de **empuje** (coste 1) e ignora los movimientos simples (coste 0):

```
# Fluente auxiliar para el coste de empujar
coste_de_empujar = Fluent('COSTE-DE-EMPUJAR', IntType(),
    p=player, pos=location, s=stone,
    f=location, t=location, d=direction)
problema_Sokoban.add_fluent(
    coste_de_empujar, default_initial_value=Int(1))
```

Ejercicio 3 — $A^* + h^{\text{máx}}$: Completa el código (2/2)

```
metrica = MinimizeActionCosts(  
    {push_to_nongoal: ___,      # coste = coste_de_empujar(...)  
     push_to_goal:    ___},    # coste = coste_de_empujar(...)  
    default=Int(0))           # resto de acciones: coste 0  
problema_Sokoban.add_quality_metric(metrica)  
  
planificador = OneshotPlanner(name='fast-downward',  
    params={'fast_downward_search_config': 'astar(hmax())'})  
resultado = planificador.solve(problema_Sokoban)  
print(resultado)
```

Objetivo: Configurar métrica para minimizar empujes y usar $A^* + h^{\text{máx}}$.

- Definir fuente `coste_de_empujar` con valor 1
- Asignar coste 1 a PUSH-TO-NONGOAL y PUSH-TO-GOAL, 0 al resto
- Configurar Fast Downward con búsqueda `astar(hmax())`

Ejercicio 3 — A^* + $h^{\text{máx}}$: Implementación

```
metrica = MinimizeActionCosts(  
    {push_to_nongoal: coste_de_empujar(  
        push_to_nongoal.p, push_to_nongoal.pos,  
        push_to_nongoal.s, push_to_nongoal.f,  
        push_to_nongoal.t, push_to_nongoal.d),  
     push_to_goal: coste_de_empujar(  
        push_to_goal.p, push_to_goal.pos,  
        push_to_goal.s, push_to_goal.f,  
        push_to_goal.t, push_to_goal.d)}},  
    default=Int(0))  
problema_Sokoban.add_quality_metric(metrica)  
  
planificador = OneshotPlanner(name='fast-downward',  
    params={'fast_downward_search_config': 'astar(hmax())'})  
resultado = planificador.solve( problema_Sokoban ) print (resultado )
```

Ejercicio 3: Conexión con la Teoría

¿Por qué $A^* + h^{\text{máx}}$?

- En Sokoban queremos un plan con **coste mínimo** (mínimo número de empujes).
- $h^{\text{máx}}$ es admisible $\Rightarrow A^*$ garantiza **optimalidad**.
- Asignar coste 0 a los movimientos del jugador y coste 1 a los empujes hace que $g(s)$ cuente únicamente los empujes realizados hasta el estado s .

Lo que hemos trabajado:

- 1 **STRIPS / PDDL**: leer y construir dominios e instancias con la biblioteca unified-planning.
- 2 **Mundo de los bloques** (Ej. 1): creación dinámica de instancias y uso de Fast Downward.
- 3 **Dominio Depot** (Ej. 2): completar las acciones de un dominio real de competición.
- 4 **Sokoban** (Ej. 3): diseñar un dominio de cero y resolver con $A^* + h^{\text{máx}}$ para minimizar el número de empujes.

Conexión con la teoría:

- Representación factorizada del estado $\leftrightarrow$ fluentes en Unified Planning.
- Listas de borrado y adición de STRIPS $\leftrightarrow$ `add_effect(h, False/True)`.
- Búsqueda con heurística $h^{\text{máx}}$ $\leftrightarrow$ relajación del borrado.
- Admisibilidad de $h^{\text{máx}}$ $\leftrightarrow$ optimalidad del plan en Sokoban.